

Game Design Document

16/01/2025

“Oil Spill Clean-up”

Authors: Jake Fitzgerald

Project Supervisor: Ross Palmer

Table of contents

Overview

- The Elevator Pitch / High Concept
- Theme, Setting and Genre
- Player Experience Goals
- View
- Targeted platform(s)
- Technical requirements(s)

GamePlay

- The First Minute (60 seconds of play)
- Game progression
- Level progression
- Objectives/Victory Conditions

Features

Game World

- Game geography
- Game World Elements: Enemies/Characters/Items

Levels

- Level description

Interface

- Controls
- In game overlays & dialogs
- HUD
- Screenflow
- Control system

AI

- Opponent AI
- Support AI

Game Art & Audio

- Audio
- Art assets
 - Characters/ animation frames
 - Items (in-game & icons)
 - Level backgrounds/maps/environment textures
 - Visual effects
 - Explosions
 - Particles
 - HUD graphics, typeface
 - Controls screen/menu/dialog backgrounds/borders/typefaces
 - Polish
 - Shaders

1. Overview

1.1. The Elevator Pitch / High Concept

A top down arcade style game where the player must navigate (interaction) a hazardous environment (obstacle) and clean-up the pollution areas and saving animals in the level (goal).

1.2. Theme, Setting and Genre

Present day in various Wexford locations (Duncannon, Rosslare Harbour, Hook Lighthouse, etc).

Genre is a top down arcade shooter.

1.3. Player Experience Goals

Fast paced action when dealing with hazards/enemies. Pacing being broken up with smaller and simple puzzle room sections both as a breather from the action and a way to give a satisfaction when solving the puzzle.

1.4. View

Top down perspective that can be static for single rooms or dynamic to allow for scrolling in large rooms.

1.5. Targeted platform(s)

PC browser game: *Firefox 134.0, Chrome 133.0*

1.6. Technical requirements(s)

Godot Engine version 4.3

2. Gameplay

2.1. The first minute (60 seconds of play)

After the Title Screen the player is loaded into the Tutorial level where they will learn the essential mechanics of how to play (*Movement, Sucking, Shooting, Dodging, picking up collectibles*). This is done from small and intuitive room design with additional help prompt text that pops up to communicate how to play. The player is loaded into a safe environment for the first couple of rooms so that they can easily learn at their own pace without fear of taking damage or restarting from dying.

2.2. Game Progression

2.2.1. The player progresses from level to level by going through the main route in each room until they reach the final room; where they need to protect a seal from a wave of hazards/enemies.

2.2.2. Difficulty is increased in gradual increments every level and is balanced with the player upgrading their character's stats.

2.2.3. Engaging rewards such as the player's score and rank that appears on an end of level screen. It allows for the player to feel accomplished in their performance and will give them a rank to make their playthrough feel unique to them. This is also to encourage players to play again for more rewards.

2.3. Level Progression

2.3.1. Continue through each room by going through a door which leads to another room.

2.3.2. Level gimmicks such as switches on floor tiles/wall tiles that can be activated by standing on them or shooting them with a projectile. Hazardous terrain tiles that must be avoided by movement or dodging through them. Certain enemies can also create these hazardous tiles.

2.4. Objectives/Victory Conditions

Player must find and rescue the seal at the end of every level to complete the game.

3. Features

3.1. Mechanics

1- Movement: Up, Down, Left, Right character movement on X,Y axes.

4 different animations for each direction with 8 frames each.

The character's collision will be handled with Godot's built-in 'Collision Shape2D', the collider shape will be a Circle so that it allows us to move along surfaces like walls smoothly without snagging onto them.

Smooth Movement:

Movement will feel smooth from responsive inputs when pressing a direction and will have subtle inertia that builds up and is released when pressing a button. This only lasts a frame where the character's speed goes from a value of 0 to then 5.0, to a base speed value of 12.0 (float), with an exponential increase. The same for when the button is released it will go from 12.0 to 5.0 and then 0 for stopping. This feels more realistic since you don't start off at full speed straight away, as well as coming to a stop instantly, this will prevent the feeling of 'jerky' motion. We can use a lerp function to calculate these values.

2 - Sucking: Can suck certain objects (props, projectiles, enemies, etc) into the character's machine to be able to then shoot back out. You suck 2 grid spaces (32 X 32 pixels) in whatever direction the character is facing (can be upgraded to 3 spaces (48 X 48 pixels)).

Sucking animation takes 4 frames in total that loop while the button is held down.

Sucking Physics:

The different entities that can be sucked up all have 2D rigidbodies. This allows us to have said rigidbody be pulled toward the X,Y location of the Player like a gravitational pull while the Suck button is held. The will be of a fast speed (15.0 pixels per second) over the distance between 16 to 32 pixels in distance (1 - 2 grid spaces).

Sucking Limits:

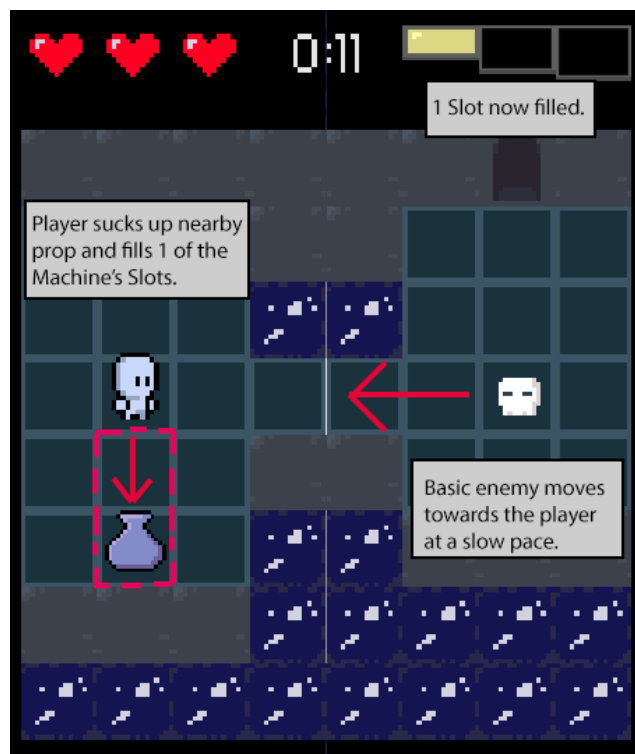
Only certain props in the environment can be sucked up, these will be made visually readable to them all being a similar colour palette (Blue). Most simple enemies can be sucked up, however certain enemies may need to be damaged initially to break their armour before being able to suck them up (to be decided).

Sucking Controls:

The button to Suck and Shoot are the same by design. This is to allow for a simple control scheme but also limit the Player to having to choose when to

Suck and when to Shoot. If Suck is held they can fill the machine up to 3 Slot Gauges, however if it is press they will only suck up a single entity and fill a single Slot Gauge. Pressing the same button when any Slot Gauge is filled will always shoot the amount, this way you can't keep shots for later you have to take the risk and use them all at once.

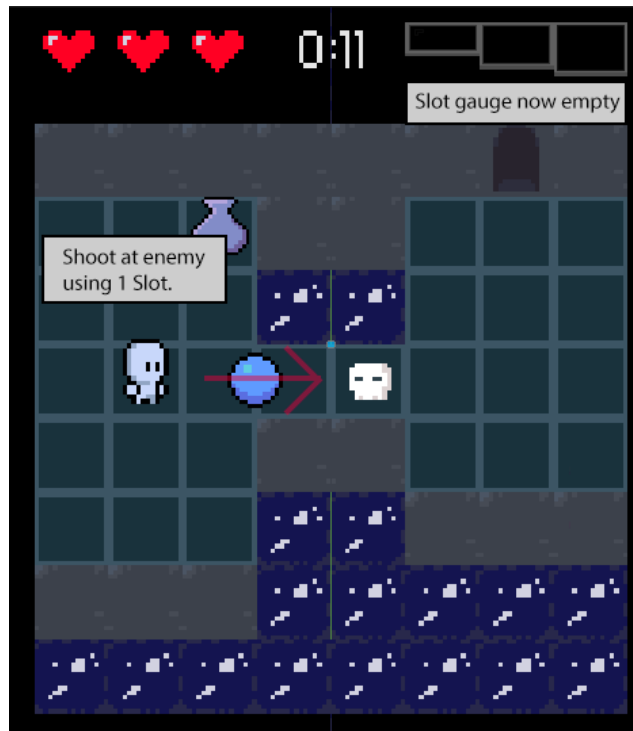
Sucking Mockup:



3 - Shooting: Can shoot out a projectile in whatever direction the character is facing. This projectile travels along a straight path until it collides with an object (enemy, prop, wall boundary, etc). Shooting animation takes 3 frames in total to allow for responsive input with no latency, with the projectile itself being spawned in front of the player on the second frame.

The projectile speed is somewhat slow being only 4 pixels per second (float 3.0) to allow for balance of the enemy movement speed and readability of which direction you have shot in.

Shooting Mockup:



Slots:

There are three slots in your machine when the game starts that can hold up to three shots. The Slot is filled instantly when sucking up an entity. By sucking up multiple entities at once when holding the suck button down, you can fill up all three slots. This means the more powerful your shot will be depending on how many slots are filled. The amount of slots can also be upgraded in the shop. [PLANNED SPRINT 1].



Shot Types:

Held shot: Hold to deplete filled slots on your machine. When shoot button is released a much stronger shot is released (3x damage with larger hitbox of 1.5 grid tiles (24 pixels) instead of regular 1 grid tile (16 pixels)). This is the default Shot Type you start the game with.

Shotgun: When at least 3 slots are filled you press the Alt Shoot button to can purchase this Shot Type at the shop. This shoots 3 shots at once depending on facing direction all at once, a shot for -45 degrees, facing direction, and +45 degrees. Each projectile travels in te same way a normal projectile would and only stops on impact with an entity or a boundary wall.[PLANNED SPRINT 2]

4 - Dodge: Can dodge certain damage-able colliders, such as hazards or enemy attacks. However you can only use a dodge when your machine is empty, meaning you can only dodge when not being able to attack (Risk vs. Reward).

Dodge moves the player 2 grid spaces in whatever direction they have pushed. Medium movement speed of 10 pixels per second.

Dodge animation has 8 frames in total.

Player has invincibility frames during this animation:

Animation has 8 frames in total, frames 2 - 6 the hurtbox collider is disabled. This allows it to be a panic button to get out of danger. There is a cooldown timer of 2 seconds to prevent the player from abusing spamming this mechanic.

5 - Item: Can use an item if you have collected or bought an item at the shop. This item is stored in your item slot which is visually shown on the HUD's UI. Different items appear in the level and are sometimes dropped from enemies that can be picked up and used. A random item appears in the shop you can buy, however it is quite expensive and you won't know what you get until you purchase it (risky). Item trigger pickup radius is 1 grid tile in size and uses a Collision Shape2D circle shape.

Item Drops:

Items also appear based on RNG when certain entities are destroyed (dropped by an enemy, prop, etc). We calculate this number based on the Player's current health (less health (a single heart) increases an item drop chance [30/70]). Full health will rarely drop an item [10/90]. There will be certain rooms like a Checkpoint Room where items will always be spawned in to help the Player, however those items will still be a randomised type.

Item types:

1 - Health Potion: Restores health by 1 chunk [COMMON]

2 - Full Health Potion: Restores health fully [RARE]

3 - Screen Wipe: Screen clearing item that destroys all enemies + breakable objects currently on screen [RARE]

4 - Invincibility: Potion (Temporary invincibility that lasts 5 seconds)

5 - Key: One per level and is used to open a bonus room that can help the player by giving them a powerful upgrade (double damage, etc) [DROPPED BY ENEMY OR FOUND AS ITEM IN SHOP]

Currency Types:

These can be used at the shop for items or upgrades.

Gem 1: value of 10

Gem 2: value of 25

Gem 3: value of 50

Taking Damage/ Invincibility Frames:

Certain states the Player will also have invincibility frames such as using an Item, Door transitions, etc. When the Player takes damage they will have invincibility frames of 2 seconds to prevent them from taking more damage during that attack animation. We will flash the Player's sprite a white colour using Godot's texture colour modifier, this visually communicates that the Player is in a invincible recovery state. We disable the Player's hurtbox collider during these times to prevent them from taking damage. There will be a small knockback effect that moves the Player in the opposite direction where they took damage (i.e. enemy attack), however it won't be too strong to knock the Player into a hazard but enough to clear them from oncoming danger. They are moved during knockbacked half a grid space (8 pixels) at a medium speed (5 pixels per second)

3.2. Visual/Audio

Art: Player character spritesheet, Enemy spritesheet, Prop spritesheet, Level sprite assets, HUD UI sprites, Menu UI Button sprites + background.

HUD Mockup:



Hitpoints represented by heart sprites in left corner, timer for duration of playtime in this level in centre, Machine's Slots gauge for how many slots are filled to allow for shooting

Each game world sprite is limited to 16 X 16 pixels. This includes the Player, Enemies, and world tiles (Ground, Walls, Traps, etc). This allows a static screen to hold 10 sprites vertically and 16 sprites horizontally. We can base many of the game mechanic's spacing and distance (Attacking, Sucking, trigger colliders size, etc) on a single tile we will call a "Grid".

The artstyle will be a simple pixel art using a limited colour palette to mimic a retro aesthetic. This lets us quickly and efficiently create art during development.

Example of artstyle:



Game: Mina the Hollower

Screen dimensions will be dynamic to be resized to a person's monitor size while in fullscreen. The game will still always try to fit in the resolution scaling of 1920 X 1080, 960 X 540, 480 X 270. This is important for playing the game inside the itch.io browser window page, since we can rescale it and keep the aspect ratio. The game can be rescaled in the Options Menu in the Main Menu.

Buttons:

Menu Buttons are scaled when the mouse cursor hovers over them and unhovers over them. A simple SFX sound is played on hover and also on click.

Sound SFX:

Player: Jump, Land, Suck, Shoot, Shoot Charged, Damage, Death, Win, Dodge, Item Use, Item pickup, Currency pickup (pitch malleable)

Level: Prop break, Door open, Currency spawn displacement sfx.

Enemy: Attack, Damage, Death, Taunt, Alert

Music BGM: Tutorial, Level 1, Game Over, Main Menu, Shop.
[All BGM tracks loop by default]

3.3. Structural Elements

Levels:

Tutorial: Teaches basics of how to play. Short and easy to complete (5 minutes max). Introduces most elements found in later levels but isolated to easily introduce to a new player the solution.

Mechanics taught: Movement, Sucking, Shooting, Dodging, Switches, Using Items, Using the Shop, Using Keys (optional)

Level 1: A challenge for the player who now understands how to play. Introduces some new elements not in the previous level but still in a casual and not too difficult approach.

Mechanics taught: More optimal ways to use Dodging (through enemy attacks, certain hazard tiles, etc). Puzzles involving more than a single Switch. New enemy types (Ranged, Exploding) and in different combinations together, instead of in isolated sections.

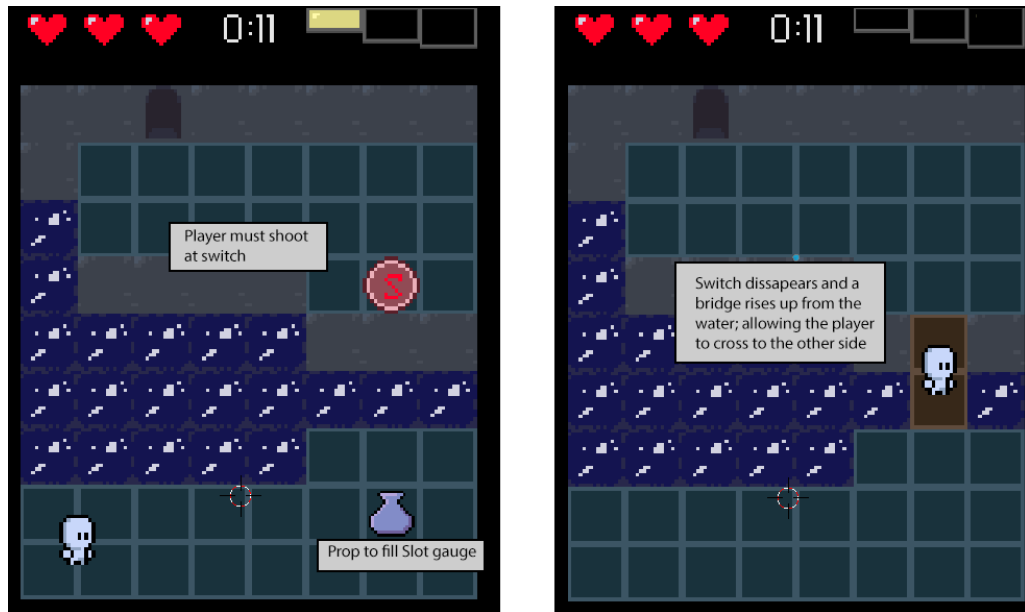
Shop: You can gain currency from destroying certain polluted objects, defeated enemies, etc which will drop small bits of rubbish and microplastics (collectible for currency).
These can be sold at the shop which is themed as a recycling centre.

The shop has a selection of three items:

1. Small Health Item
2. Full Health Item
3. Key Item

Level Gimmicks:

Wall Switch Mockup:



Standing Switch: This can be shot at and will then send a signal to any entity listening for it in that map.

Floor Switch: Works similar to the Wall Switch except requires the Player stepping on it's tile in the level for it to activate (boolean) which causes a different effect elsewhere in the level (Raising a bridge, opening a door, activating a trap to damage enemies, etc)

3.4. Game Visual Elements

Shaders: [See 8.5 for full Shader breakdown]

Shaders will be used to convey different visual effects such as a vignette, outline and water reflection.

3.5. Interactable Entities

Doors: This entity is used for transition from different gameplay scenes. It can be unlocked with key (optional depending on room). It has a trigger collider that when the Player enters will call the Transition Fade effect, when this effect has ended it then loads a determined scene. This is set in the editor with a string that is the local path to the scene in the game's directory. This makes it very easy to change what scene it should transition to in the editor.

Signs: These can be interacted with using the interact common ('E' key) and display a visual box on the screen containing a hint or tutorial on how to play. A sign's text is loaded in from a JSON file that is stored in the Dialogues folder in the game's directory.

4. Game World

The game takes place on a section of Duncannon Beach in Wexford. An oil tanker has crashed along the shore and has spilled oil along the coast. The player controls a character in charge of disposing of the oil meaning they will have to clean the different areas of the Duncannon's coastline. The tutorial level takes place along the beach and will require not many tilesets to represent the location. The second level will be set inside the oil tanker itself, which will not be able to reuse any of the tilesets/ several assets from the tutorial level.

5. Levels/Rooms

There will be many rooms to distinguish each section for the different levels. For this project the focus will be on the tutorial level which will contain 8 distinct rooms.

Room List:

- Movement Introduction
- Suck/ Shoot/ Wall Switch Introduction
- Ranged Enemy/Destructable Wall/ Chest/ Key Introduction
- Shop Room
- Seal Lead to Exit Room (A* Showcase)
- Dodge Room (Boids Showcase)
- Water Reflection Room
- Seal protect Room (Final)

Movement Introduction

First gameplay after pressing Start Button in the Main Menu. The goals for the Player is to learn how to move and getting comfortable with how the game controls. They can interact with a sign in the centre of the map that tells them the controls for moving and interaction. There are a total of 7 gems in this map and are easy to collect since they encourage basic movement. They can leave the map and progress to the next room by using the Door at the top of the map (Player Spawn is at the bottom of the map).



Suck/ Shoot/ Wall Switch Introduction

The Player must cross a gap of water place in the centre of the map by shooting at a switch they cannot reach. There is a sign placed near an area where they must shoot the switch that explains how to suck up enemies and shoot with what keys. They must suck up the oil enemies placed away from the spawn to fill up their Slot-Gauge. They can then shoot the standing switch that activates a bridge to raise up, allowing them to cross over the gap of water and reach the Door on the other side, bring them to the next room.



Ranged Enemy/Destructable Wall/ Chest/ Key Introduction

This room teaches the player about ranged enemies that can shoot from a large distance in a pattern that you must avoid. The player can find an optional key and locked chest that will reward them with several randomised items to help with the overall difficulty curve for the level as a whole. There a sign directly ahead of the Player Spawn that they can read which will give them hints about a destructable wall they can break by shooting at it. The

player can easily see this due to the cracked pattern on the sprite of the destructable wall.

There are two simple oil enemies placed near it so they can quickly destroy the wall and find a secret area that is revealed when entering it's trigger collider. They will find a chest that is locked, they need to explore the rest of the map to find the key then backtrack here for the reward. It is important to introduce the locked chest before the key as to create the want and need to solve this puzzle. Later on in the level there is a sign explaining that some enemies cannot be sucked up due to them being armoured so they need to be shot at instead (Ranged Enemies).



Shop Room

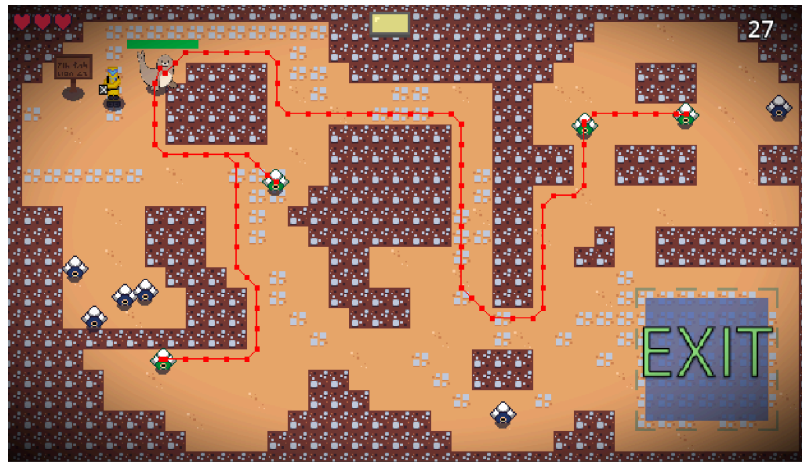
Simple room containing a Shopkeeper NPC that explains how to purchase items. The items themselves are locked in a cabinet that the player must spend their currency to open. The cost for the item is displayed above the item when inside it's trigger collider.



Seal Lead to Exit Room (A* Showcase)

The player must help a Seal NPC navigate a maze like room to an Exit area. When the Seal NPC is in this Exit area they move onto the next room. The Seal NPC uses a custom map A* pathfinding to find the Player's location and navigate around obstacles. The player must protect the Seal NPC who takes damage from colliding with any enemy (Seal NPC has 8 hitpoints). There are new enemies introduced in this section who use A* pathfinding with a

Navigation Agent 2D to find and move to the Seal NPC's position. There are simple oil enemies in the map to suck up to allow you to shoot the Navigation based enemies.



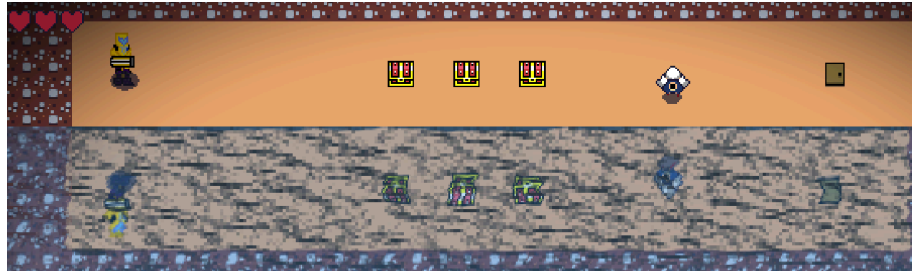
Dodge Room (Boids Showcase)

This room is purely to showcase the Boids flocking algorithm. There is very little gameplay in this scene and is mainly used visually.



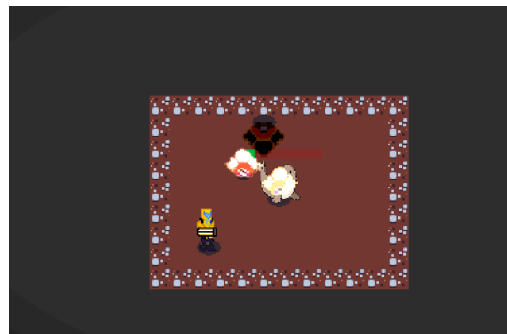
Water Reflection Room

This room showcases the Water 2D shader that reflects a masked area of the screen. This room also contains three chests to give the player more health and a certain amount of currency for use in the next level. A simple enemy is here primarily to show a moving spritesheet in the reflection shader but also helps fill the Slot-Gauge for the Boss room that follows next.



Seal protect Room (Final)

The player must protect the Seal NPC that is placed in the centre of the map. There are enemies spawned offscreen that travel at a randomised speed within a clamped range (10.0 - 30.0 [10 to 30 pixels per second]) towards the position of the Seal NPC. There is a Boss enemy that travels along a predetermined spline that curves around the map. The player must suck up the simpler enemy and shoot them at the Boss enemy before it reaches the Seal NPC (takes 5 hits to destroy the Boss). The Seal NPC in this scene is a different entity that is static since it does not need to move but also has more hitpoints (10) to make it a balanced and fair challenge.



7. AI

7.1. Enemy AI

1 - Ground Slime: Simple enemy that moves towards the player. Has a very small hitbox that can damage the player however has a large hurtbox which allows the player to suck these enemies into the machine with ease. This allows these enemies to be used as ammunition for the player in most scenarios. However when they swarm then they can become much more of a threat and need an area of attack shot to deal with (Charged Shot, Shotgun Shot, etc).

Movement:

Moves at a medium pace when farther away from the player at 10 pixels per second. They have two different AI states (Idle, Alert), when the Player has entered their alert trigger they will be in their Alert state and will remain in this state until they are reset by the Player leaving the current room and re-entering or by the Player dying and the level being reset.

In their Idle state they will move about the level in a random pattern, when they are Alert they then move towards the Player's current position in the level. Their pathing is limited to the Navmesh of that room, this is to prevent them from falling into certain hazards like water or getting stuck in tight spaces.

2 - Ranged Slime: Aims at player when inside its alert radius (circle collider trigger). When inside this trigger a timer will countdown from 4 seconds, then the enemy will shoot at the player's current position on the level. Then a cooldown timer will become active, when this ends the enemy can again aim and attack based on the attack timer. [PLANNED SPRINT 2]

These enemies do not move and their alert trigger is usually large enough so that they will start to aim at Players when they are in a regular sized room (16 X 10 grid size (256 X 160 pixels).

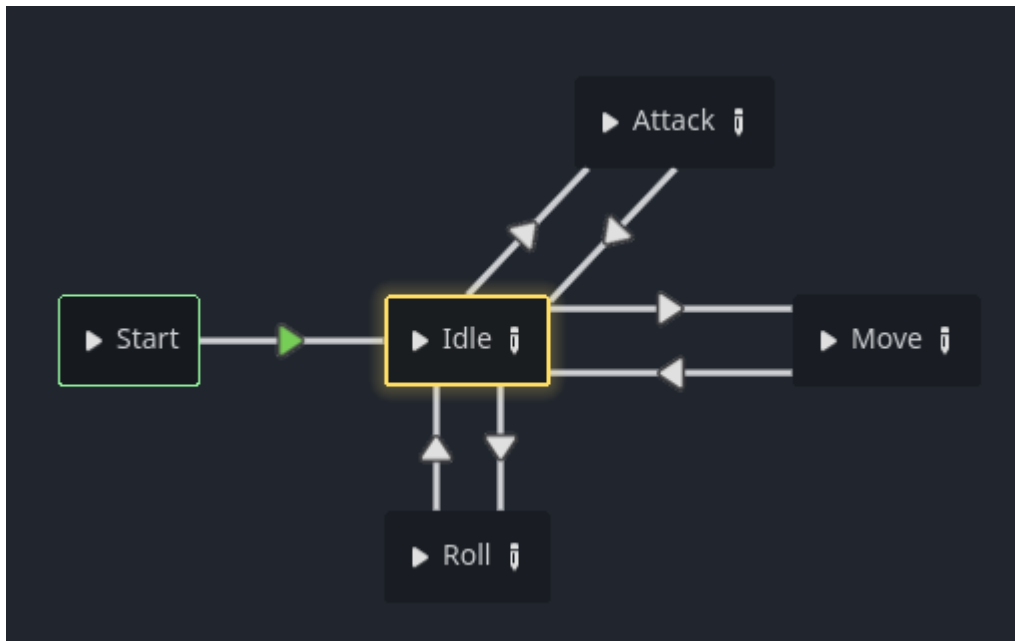
3 - A* Pathfinding Enemy: This enemy uses the A* star grid to navigate the obstacles in the maze room towards the position that the Seal NPC is standing on. It takes in a weight cost for each tile that is defined in a custom layer, where each of the sprite's coordinates are flagged for having different weight costs, affecting which tiles will be adding into the path array that the enemy takes when moving.

8. Game Art & Audio

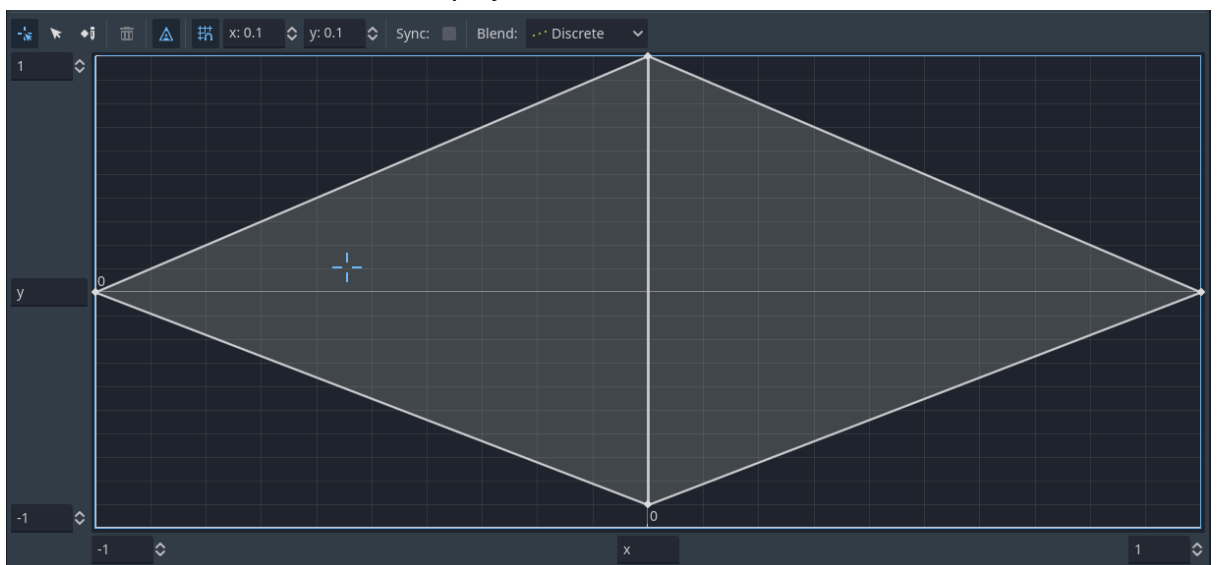
8.1. Animation Blend Tree

The Player character uses an animation blend tree for transitioning to different directional states within the spritesheet. We use an AnimationTree node in combination with a Animation Player node.

State Machine for animation transitions



Blend Tree that is moved depending on the input vector for which directional the player moves in



8.2. Sound Manager

Sound Manager is static autoload node that is present in every scene in the game. It contains every audio file (Music/SFX) for the game in separate `AudioStream2D` nodes. This allows us to manage, modify and replace sounds during development quite easily.

8.3. Audio Mixer Buses

We have separate mixers for different audio types (Master, Music, SFX). Both the Music and SFX bus are linked to the Master bus, meaning if you alter the

Master bus (Volume, Effect, Mute), then the two dependent buses will be changed aswell.

8.4. Polish

Gameplay Effects:

Hit-stop:

Game freezes on the current frame for for a certain amount of frames to sell the effect of an impact. This reuses similar code to how the Pause feature works where all active entities and input is disabled. This helps gives a better sense of impact for Player damage, Enemy damage, Player Death, certain rare pickups on collection.

We set three floats as the values that freeze the game engine's processor, this gives us flexibility for how much impact we want depending how much damage was taken (Player hit is short, while Boss death is long) :

- hitstop_short_amount = 0.10
- hitstop_medium_amount = 0.25
- hitstop_long_amount = 0.50

Screenshake:

Screen canvas moves in a random noise pattern to simulate camera movement to convey Player damage or an explosion effect. Can be set to different movement and speed amounts based on damage taken. Shake speed is a float value that takes an amount. Shake movement amount is based within a range on X/Y axis (Example: (3,3)) as vector2 coordinates.

The noise pattern values are generated in a function generate_noise() that returns an array of vector2. We then use the unique vector2 in the array as points that the camera origin will be moved to. The camera movement will be smoothed by lerping the velocity as it travels from point to point.

Small [1 hitpoint]:

- Movement Range (1.5, 1.5)
- Speed [5.0]

Medium [2 hitpoint]:

- Movement Range (3, 3)
- Speed [8.0]

Large [3 hitpoint]:

- Movement Range (5, 5)
- Speed [12.0]

Trails effect:

Trails that are created behind a moving Projectile. This is created using a built-in function in Godot to generate a “PackedVector2Array”, that is a series of points in an array to draw simple square shapes in a row. We can apply a curve to this shape to make it get narrower along its length, as well as a gradient colour so that it starts off opaque and gradually becomes partially transparent.

We use this effect for the Player’s Projectile. When the projectile is fired, we preload the Trail scene and make an instance of it in the current scene tree and attach it to the Projectile as a child node. This trail gets deleted when the projectile parent is deleted from colliding with a Wall or an Enemy.

Screen Fade Effect:

A simple fade to black, or fade to transparent screen wipe effect for transitioning between scenes (both level and menu scenes). This is done using a simple colour rectangle that covers the screen resolution and two animations (fade_to_black, fade_to_transparent). For the first we animate the colour rectangle’s colour from fully transparent to fully opaque by setting two keyframes at 0 alpha and the other at 255 alpha.

We make this scene an autoloading singleton, this way it’s preloaded into every other scene in the game and can easily be called using a function called `_on_animation_finished()`. In this function we did a simple check to see what animation is currently playing, if it’s “fade_to_black” then we emit a signal called “_on_transition_finished”, so we can await the signal in scripts where we called the function so it plays the entire fade effect before loading the next scene. If it’s “fade_to_transparent” then we set the colour rectangle to be invisible.

Particle List:

These particles are mostly created by playing an animated spritesheet that is spawned into the current scene tree and is deleted when finished.

Generic Damage particles:

Spawn based on a generic damage effect such as when any Projectile collides with a wall.

Player Take Damage particles:

Spawn at the Player's location when damage is taken.

Enemy Take Damage particles:

Spawn at the Enemy's location when damage is taken.

Pickup particles:

Spawn at the pickup's location when item is collected.

Shoot particles:

Spawn at the entities shoot position for when a Projectile is fired.

Chest Open particles:

While the other particles were created using an animated spritesheet, this was created using Godot's built-in particle generator called "CPUParticles2D". We create a spherical shape so that the particles they spawn from the centre of the Chest when it is playing it's "Open" animation and move outwards. We also randomise each particle's modulate colour since each item has a different colour. This helps sell the effect of different items spawning from the chest into the scene.

8.5. Shaders

*Shader List:***Outline Shader:**

Per object effect that draws an outline around the sprite. This is used to convey if the Player is within

Vignette Shader

Covers the entire screen and is the first layer to be drawn in the render order layers. This shader is applied to a material assigned to a colour rectangle that can be placed into any scene.

Blur Shader:

Used for the Pause Menu to blur the entire canvas layer that sits behind the UI overlay. Acts similar to Gaussian Blur where each pixel is blurred into it's neighbour to create large blobby shapes.

Water/Reflection Shader:

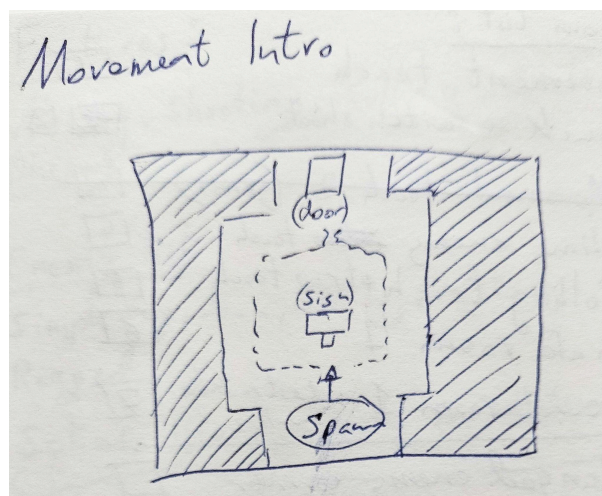
- Reflection by flipping UV on Y axis
- Mask the area where the Water tiles are to separate the shader from the rest of the screen elements (Player, Enemies, HUD, etc)
- Water texture that scrolls
- Blurring the reflected sprites (similar to the Blur Shader)

Reference image :



9.0 Layout sketches

Initial layout drawings for how each level will play and other explored ideas before creating them in the editor.



Suck + shoot Intro

